

Architektura web + mobile: mapa wód PZW Okręgu w Rzeszowie

Stan na 20 września 2026 · autor: Kamil Kornacki (z pomocą Claude) · wersja 1.3 — dane z BDOT10k naniesione

W jednym zdaniu: aplikacja webowa już istnieje; najtańszą i najbezpieczniejszą drogą do telefonu jest zrobienie z niej PWA na obecnym stosie, z jedną bazą danych dla obu, a wejście do sklepów (Capacitor) zostawić jako opcjonalny, płatny etap dla Okręgu. Łączny nakład Twojego czasu do domknięcia projektu bez sklepów: **ok. 120–190 godzin**, ze sklepami **ok. 160–250 godzin** (sekcja 12).

1. Punkt wyjścia i wymagania
2. Jedna warstwa danych, dwa „opakowania”
3. Warianty aplikacji mobilnej
4. Porównanie wariantów
5. Tryb offline nad wodą
6. Panel operatora na telefonie
7. Aplikacja webowa: build czy bez
8. Koszty roczne w złotych
9. Bezobsługowość i przekazanie PZW
10. Rekomendacja i plan etapów
11. Decyzje do podjęcia
12. Koszt czasowy: ile godzin od Ciebie
13. Źródła

1. Punkt wyjścia i wymagania

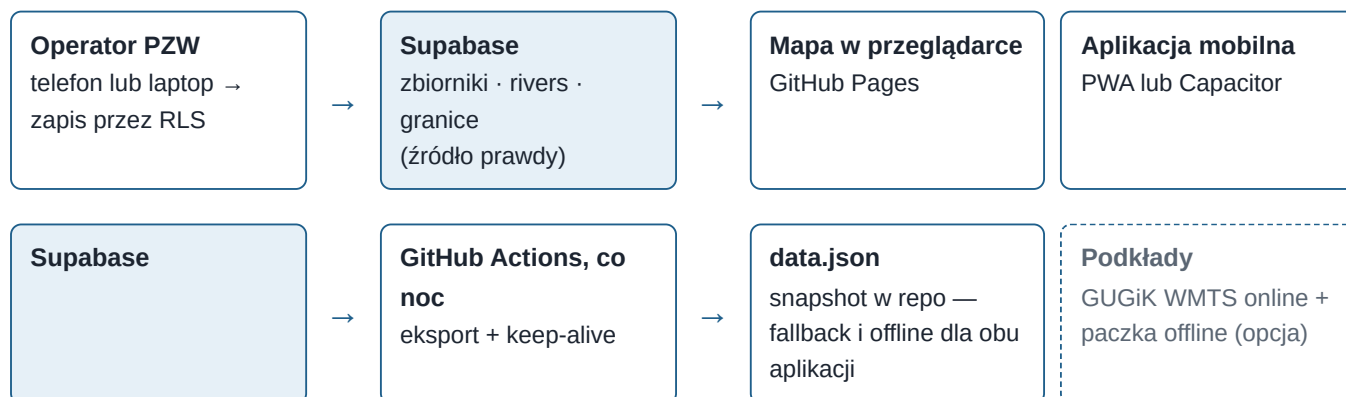
Aplikacja webowa już istnieje i ma architekturę, która nadaje się na fundament dla mobile: statyczny front (Leaflet 1.9.4, vanilla JS, bez kroku budowania) na GitHub Pages, dane w Supabase (tabele `zbiorniki`, `rivers`, `granice`, RLS z allow-listą operatorów) z awaryjnym `data.json`, oraz panel operatora `admin.html` z edycją punktów i geometrii rzek (Leaflet-Geoman). Repo: `korntech/mapa-wod-pzw-rzeszow`, mapa: `korntech.github.io/mapa-wod-pzw-rzeszow`.

Pytanie nie brzmi więc „jak zbudować dwie aplikacje”, tylko „jak z jednej bazy kodu i jednej bazy danych obsłużyć przeglądarkę i telefon, nie mnożąc rzeczy do utrzymania”.

Wymaganie	Ustalenie
Zakres mobile v1	Pełna mapa (wyszukiwarka, filtry, „najbliżej mnie”, nawigacja) plus panel operatora — operator PZW poprawia lokalizację z telefonu w terenie
Offline	Przeglądanie łowisk i podkładu bez zasięgu nad wodą; edycja może wymagać sieci
Utrzymanie	Nikt techniczny po stronie PZW — system ma być bezobsługowy, bez serwerów, bez rzeczy wygasających co miesiąc
Koszt	Jak najbliżej 0 zł/rok; każda pozycja płatna musi być uzasadniona i możliwa do opłacenia przez Okręg
Własność	Docelowo przekazane Okręgowi PZW w Rzeszowie; kod otwarty (MIT), wkład społeczności przez GitHub
Kompetencje	Autor zna HTML/JS; brak doświadczenia w aplikacjach mobilnych — krzywa nauki jest kosztem
Sklepy	Decyzja po porównaniu (sekcje 4, 8 i 12)
Źródła danych	Wyłącznie polskie źródła państwowe (GUGiK/PZGiK) — decyzja z 20.09.2026

2. Zasada nadrzędna: jedna warstwa danych, dwa „opakowania”

Niezależnie od wyboru technologii mobilnej, obie aplikacje czytają i piszą tę samą bazę — nie budujemy drugiego backendu dla telefonu. Różnica między webem a mobile sprowadza się do tego, w czym uruchamia się ten sam kod frontu i skąd bierze podkład mapy, gdy nie ma sieci.



Odczyt: Supabase, a gdy nie odpowiada — snapshot `data.json`, który nocny job w GitHub Actions eksportuje z bazy i commituje do repo. Dzięki temu snapshot jest zawsze świeży (dziś jest ręczny i może się rozjechać z bazą), a mapa działa nawet gdy baza jest zapauzowana. Ten sam job wykonuje zapytanie do bazy, co liczy się jako aktywność i zapobiega pauzowaniu projektu Free po tygodniu bezczynności. Warstwa przestrzenna w tej bazie pochodzi już w całości z BDOT10k: przetworzono paczki z 23 powiatów, czyli 23 935 poligonów wód i 85 216 odcinków cieków.

Wariant backendu	Opis	Koszt/rok	Kiedy wybrać
A. Supabase Free + snapshot w repo (rekomendowany)	Obecna baza i RLS bez zmian; dochodzi nocny eksport do <code>data.json</code> i keep-alive. Limity Free (500 MB bazy, 5 GB transferu/mies.) są o rzędy wielkości powyżej potrzeb: całe dane to ok. 56 KB	0 zł	Domyślnie — spełnia „bezobsługowość”, bo awaria bazy degraduje do statycznej mapy, nie do białej strony
B. Git jako baza (bez Supabase)	Panel operatora zapisuje zmiany jako commit do repo przez GitHub API; mapa czyta tylko <code>data.json</code>	0 zł	Gdyby PZW nie chciało żadnego zewnętrznego konta poza GitHub. Minus: operator potrzebuje konta GitHub i tokenu; historia zmian w Git (plus), ale UX edycji i uprawnienia trudniejsze dla osób nietechnicznych
C. Supabase Pro	Jak A, plus codzienne backupy, brak pauzowania, wsparcie e-mail	ok. 1 140 zł	Tylko jeśli Okręg chce płacić za gwarancję; przy wariantcie A snapshot w Git pełni rolę backupu za darmo

Dwie zmiany w modelu danych warto zrobić teraz, bo mobile je wymusi: kolumna `updated_at` już jest — dołożyć zapytanie „co zmieniło się od daty X” (synchronizacja przyrostowa zamiast pobierania wszystkiego) oraz tabelę `zgłoszenia` (wędkarz zgłasza błąd w danych z telefonu; operator zatwierdza w panelu). Druga jest opcjonalna, ale to najtańszy sposób, by społeczność poprawiała lokalizacje `a:1` bez dawania jej praw zapisu.

3. Warianty aplikacji mobilnej

Cztery realne drogi, od najtańszej do najdroższej w nauce i utrzymaniu. W1 i W2 używają istniejącego kodu; W3 i W4 oznaczają drugą bazę kodu.

W1. PWA — obecna strona jako aplikacja instalowana ze strony

Do istniejącego `index.html` dochodzi manifest, service worker (cache aplikacji i danych) i magazyn offline (IndexedDB). Użytkownik dodaje mapę do ekranu głównego z Safari lub Chrome; dostaje ikonę, pełny ekran i pracę offline. Panel operatora to ta sama strona (`admin.html`), więc „wersja mobilna panelu” to głównie praca nad układem dotykowym.

- **Plusy:** 0 zł, jedna baza kodu, jeden deploy (push do `main`), żadnych kont deweloperskich ani przeglądów w sklepach, aktualizacja natychmiastowa u wszystkich.
- **Minusy:** brak wpisu w App Store/Google Play (instalacja przez „Udostępnij → Do ekranu początkowego”, którą trzeba wytłumaczyć wędkarzom); na iOS brak automatycznej zachęty do instalacji i brak pracy w tle (np. śledzenie GPS przy wygaszonym ekranie).
- **Co trzeba umieć:** to, co już jest w projekcie (JS, Leaflet) plus service worker i IndexedDB — kilka dni nauki.
- **Stan iOS:** zainstalowana PWA jest zwolniona z 7-dniowego kasowania danych, które dotyczy zwykłych stron; ma taki sam limit pamięci jak Safari (do 60% dysku na źródło); w UE działa normalnie po tym, jak Apple wycofało się z jej blokowania w iOS 17.4.

W2. Capacitor — ten sam kod w natywnej powłoce, w sklepach

Capacitor pakuje istniejącą stronę w natywny projekt iOS i Android. Kod webowy zostaje jeden; dochodzą wtyczki (geolokalizacja, system plików na paczkę kafli offline, ewentualnie powiadomienia). Buduje się lokalnie: Xcode 26 na Macu (masz Maca) i Android Studio.

- **Plusy:** wpis w sklepach, ikona i instalacja „jak prawdziwa aplikacja”, dostęp do natywnych API, wciąż jedna baza kodu — naturalny krok po W1, bez przepisywania.
- **Minusy:** konta deweloperskie (Apple 99 USD/rok, dla organizacji non-profit możliwe zwolnienie; Google 25 USD jednorazowo), przeglądy aplikacji, coroczna „higiena sklepowa”: nowe wymagane wersje SDK, certyfikaty, formularze prywatności. To stałe zadanie techniczne — sprzeczne z „nikt techniczny po stronie PZW”, chyba że robi to społeczność.
- **Co trzeba umieć:** podstawy Xcode/Android Studio i proces publikacji; sam kod aplikacji bez zmian.

W3. React Native z Expo — natywna aplikacja z JavaScriptu

Nowa aplikacja w React Native, mapa przez `react-native-maps` lub MapLibre, dane z tego samego Supabase. Expo daje budowanie w chmurze (EAS: 15 buildów iOS i 15 Android miesięcznie za darmo) i aktualizacje bez sklepu.

- **Plusy:** najlepsze wrażenia „natywności”, płynna mapa wektorowa, praca w tle, duża społeczność.
- **Minusy:** druga baza kodu (web zostaje osobno), nauka Reacta, RN i ekosystemu Expo od zera, te same koszty sklepów co W2, więcej zależności do aktualizowania co roku. Dla projektu o 90 obiektach na mapie to przerost.

W4. Flutter — natywna aplikacja w Dart

Analogicznie do W3, w języku Dart i z widgetami Fluttera (mapa np. `flutter_map` lub MapLibre). Może też budować wersję webową, ale mapowe strony Fluttera na webie są cięższe od Leafleta i gorsze dla wkładu społeczności przyzwyczajonej do HTML/JS.

- **Plusy:** jeden język na iOS, Android i (teoretycznie) web; bardzo dobra wydajność.
- **Minusy:** nowy język i ekosystem, koszty sklepów, wyższy próg wejścia dla kontrybutorów z PZW/społeczności niż HTML/JS.

4. Porównanie wariantów mobilnych

Przy wymaganiach „bezobsługowość, 0 zł, brak doświadczenia mobilnego” W1 wygrywa w każdej kolumnie poza obecnością w sklepach; W2 jest jedyną drogą do sklepów, która nie wymaga drugiej bazy kodu.

Kryterium	W1 PWA	W2 Capacitor	W3 React Native/Expo	W4 Flutter
Koszt stały (zł/rok)	0	ok. 380 (Apple) + 95 jednorazowo (Google); 0 przy zwolnieniu non-profit	jak W2	jak W2
Nakład na v1 (godziny, szacunek)	15–30	W1 + 20–40 na powłokę i publikację	120–250	120–250
Krzywa nauki	Mała: service worker, IndexedDB	Średnia: Xcode, Android Studio, proces sklepów	Duża: React, RN, Expo	Duża: Dart, Flutter
Bazy kodu	1	1	2	2 (lub 1 z gorszym webem)
Aktualizacja u użytkownika	Natychmiast po deployu	Przez sklep (przegląd 1–7 dni) lub live-update	Przez sklep lub EAS Update	Przez sklep
Praca offline (dane + podkład)	Tak, w limitach pamięci przeglądarki	Tak, pełny system plików	Tak	Tak
GPS w tle, powiadomienia push	Nie (iOS) / częściowo (Android)	Tak	Tak	Tak
Obecność w sklepach	Nie	Tak	Tak	Tak
Coroczne obowiązki techniczne	Brak	Podbicie SDK, certyfikaty, formularze sklepów	jak W2 + zależności RN	jak W2 + zależności Fluttera
Ryzyko dla PZW bez informatyka	Niskie	Średnie: aplikacja może wypaść ze sklepu, gdy nikt nie podbije wersji	Wysokie	Wysokie
Wkład społeczności	Łatwy (HTML/JS, PR do repo)	Łatwy w kodzie, trudny w publikacji (dostęp do kont)	Trudniejszy	Trudniejszy

Oplaty: Apple Developer Program 99 USD/rok; zwolnienie z opłaty dla organizacji non-profit wymaga podmiotu prawnego z wpisem w rejestrze (nie osoba prywatna). Google Play 25 USD jednorazowo; konto organizacji wymaga numeru D-U-N-S i dokumentów rejestrowych, za to nie obowiązuje go test z 12 testerami przez 14 dni, który dotyczy kont osobistych. Przeliczenie po kursie NBP 3,80 zł/USD z 18.09.2026.

5. Tryb offline nad wodą

Offline składa się z trzech niezależnych warstw; pierwsze dwie są tanie i pewne, trzecia (podkład mapy) jest jedyną prawdziwą decyzją projektową.

Warstwa	Rozwiązanie	Rozmiar	Uwagi
Kod aplikacji	Service worker, strategia „cache najpierw, odśwież w tle” (np. Workbox)	ok. 1 MB z Leafletem	Biblioteki z CDN trzeba przenieść do repo, inaczej nie da się ich zapisać w cache
Dane łowisk	Pełny zestaw w IndexedDB, odświeżany z Supabase przy starcie (przyrostowo po <code>updated_at</code>)	ok. 60 KB	Działa identycznie w PWA i Capacitorze
Podkład mapy	Do wyboru: (a) cache kafli GUGiK dla okolicy, (b) paczka wektorowa PMTiles, (c) bez podkładu offline	(a) 20–100 MB na okolicę łowiska; (b) 100–400 MB na Podkarpacie — szacunek, do zmierzenia	Patrz niżej

Podkład offline — trzy opcje

(a) Cache kafli rastrowych GUGiK na żądanie. Użytkownik na liście łowisk wciska „zapisz okolicę offline”; aplikacja pobiera kafle WMTS dla promienia np. 3 km w zoomach 12–16 i trzyma je w Cache API. Zero nowej infrastruktury, ten sam Leaflet, podkład topograficzny BDOT10k, który już jest w projekcie. Minus: kafle rastrowe są ciężkie, więc „cały okręg offline” nie wchodzi w grę, a na iOS duży cache może zostać wyczyszczony przy braku miejsca na telefonie. Trzeba też zweryfikować regulamin usług GUGiK pod kątem lokalnego buforowania — dane są otwarte, ale warto mieć to na piśmie. Fallback kafli OpenStreetMap został usunięty z projektu, więc tryb offline korzysta wyłącznie z kafli GUGiK — i to akurat upraszcza sprawę od strony formalnej: polityka korzystania z kafli OSM wprost zabrania pobierania ich na zapas, a w przypadku usług GUGiK takiego zakazu nie ma.

(b) Paczka wektorowa PMTiles + MapLibre GL. Jeden plik z kaflami wektorowymi dla obszaru okręgu, hostowany statycznie na Cloudflare R2 (10 GB i ruch wychodzący za darmo) albo w GitHub Releases. Online mapa czyta z niego fragmenty przez HTTP range requests; offline użytkownik pobiera cały plik raz. Skoro projekt przechodzi na źródła polskie, taka paczka musiałaby powstawać z BDOT10k, a nie z OpenStreetMap — co oznacza własny proces budowania kafli (ekstrakt warstw, generalizacja, tilowanie, aktualizacja przy każdym nowym wydaniu danych), czyli istotnie więcej pracy niż pobranie gotowej paczki z zewnętrznego serwisu. Plusy: cały okręg offline, płynne obracanie i skalowanie, mały transfer online, dane z tego samego źródła co reszta projektu. Minusy: zamiana Leafleta na MapLibre GL (Geoman w panelu operatora nie działa z MapLibre — panel musiałby zostać przy Leaflecie albo użyć innego edytora geometrii), własny pipeline kafli do zbudowania i utrzymania, coroczne odświeżenie paczki jako zadanie w GitHub Actions. Z tego powodu opcja pozostaje odłożona.

(c) Offline tylko dane, bez podkładu. Pinezki, granice i linie rzek na pustym tle z siatką współrzędnych, plus „Najbliżej mnie” z GPS. Brzmi ubogo, ale odpowiada na pytanie „gdzie jest granica obwodu i jakie są zasady”, które jest głównym powodem offline. Koszt: zero.

Rekomendacja: zacząć od (c) jako gwarantowanego minimum, w tym samym etapie dodać (a) jako opcję „zapisz okolicę”, a (b) rozważyć dopiero, gdy wędkarze zgłoszą, że chcą całego okręgu offline. Taka kolejność nie wymaga zmiany biblioteki mapowej ani panelu operatora.

Ograniczenia iOS istotne dla decyzji: brak synchronizacji w tle (dane odświeżają się tylko przy otwartej aplikacji), pytanie o zgodę na lokalizację może wracać, cache może zostać wyczyszczony przy braku miejsca — stąd dane łowisk zawsze dołączamy też do samej aplikacji (snapshot `data.json`), żeby nawet po wyczyszczeniu cache mapa miała co pokazać.

6. Panel operatora na telefonie

Panel `admin.html` technicznie już otwiera się na telefonie i zapisuje przez Supabase Auth, więc „panel w mobile” to praca nad ergonomią dotyku, nie nowa funkcja. W v1 edycja wymaga sieci; przeglądanie w panelu działa offline tak jak w mapie publicznej.

Element	Dziś	Do zrobienia na telefon
Przeciąganie pinezki	Działa myszką	Tryb „ustaw tu”: wybierz łowisko, wycentruj mapę celownikiem, zatwierdź przyciskiem — pewniejsze niż drag palcem; przycisk „użyj mojego GPS” dla operatora stojącego nad wodą
Edycja rzek (Geoman)	Wierzchołki na desktopie	Wierzchołki są za małe pod palec: powiększyć uchwyt, ograniczyć mobile do przesuwania pojedynczych wierzchołków i „wstaw punkt tutaj”; rysowanie całych nowych odcinków zostawić na desktop
Formularz atrybutów	Panel boczny	Dolny arkusz (bottom sheet) przewijany, pola z odpowiednimi klawiaturami
Logowanie	E-mail + hasło	Bez zmian; opcjonalnie link magiczny na e-mail (Supabase to wspiera) — wygodniejsze na telefonie niż hasło
Konflikt edycji	Brak zabezpieczenia	Porównanie <code>updated_at</code> przed zapisem: jeśli ktoś zmienił rekord w międzyczasie, pokaż różnicę zamiast nadpisywać

Kolejka offline dla edycji (zapis lokalnie, wysłanie po odzyskaniu sieci) jest wykonalna, ale wprowadza konflikty i stany pośrednie, które trudno wyjaśnić operatorowi nietechnicznemu. Prostsza alternatywa o tym samym efekcie w terenie: operator zapisuje w telefonie notatkę „pozycja GPS + nazwa łowiska” jako zgłoszenie (tabela `zgłoszenia` z sekcji 2), a właściwą edycję zatwierdza później, mając sieć. Zgłoszenie wysyła się samo, gdy tylko wróci zasięg.

Panel nie musi przy tym kazać operatorowi szukać akwenu na mapie. Mając poligony wód z BDOT10k (warstwa `OT_PTWP_A`, z konturem i powierzchnią), system sam proponuje 3–4 kandydujące akweny w okolicy przybliżonej pinezki i pokazuje przy każdym powierzchnię porównaną z powierzchnią z wykazu — operator tylko wybiera właściwy. Praca przy jednym zbiorniku skraca się z minut do sekund. Z 49 zbiorników 39 (80%) miało lokalizację przybliżoną — pinezka stała w środku miejscowości, nie na akwenu — ale 10 z nich automat przeniósł już na rzeczywiste akweny bez udziału człowieka. Zostaje 29 pozycji, dla każdej z nich lista kandydatów jest przygotowana; 4 z tych 29 nie mają dobrego kandydata w promieniu 5 km i wymagają wiedzy lokalnej. Uczciwe zastrzeżenie: to nie jest pełna automatyzacja — część pozycji z wykazu to kompleksy kilku stawów i wymagają decyzji człowieka, a nie dopasowania po powierzchni.

7. Aplikacja webowa: bez builda czy z lekkim buildem

Zostać przy vanilla JS, ale wprowadzić minimalny krok budowania (Vite) — to jedyna zmiana, która jest warunkiem PWA i Capacitora, a nie psuje prostoty dla kontrybutorów.

Dziś projekt to trzy pliki HTML z bibliotekami z CDN. Dla PWA biblioteki muszą być w repo (cache offline), kod wspólny mapy i panelu warto wydzielić do modułów (`data.js` , `map.js` , `offline.js`), a Capacitor oczekuje katalogu z gotową stroną. Vite robi dokładnie to: `npm run build` składa moduły i zależności do folderu `dist/` , który GitHub Actions publikuje na Pages, a Capacitor pakuje w aplikację.

Opcja	Plusy	Minusy
Zostawić jak jest (bez builda)	Zero narzędzi, każdy edytuje HTML	Brak offline z bibliotekami z CDN, duplikacja kodu między <code>index.html</code> i <code>admin.html</code> , brak wspólnego kodu z Capacitorem
Vite + moduły ES, bez frameworka (rekomendowane)	Jeden <code>npm install</code> && <code>npm run dev</code> ; Leaflet, Geoman i supabase-js z npm; wspólne moduły; wtyczka PWA generuje service worker	Kontrybutor musi mieć Node; GitHub Actions zamiast „push do main = deploy” (i tak potrzebne dla snapshotu)
Framework (React/Vue/Svelte)	Komponenty, stan	Nowa nauka, więcej zależności do aktualizowania — bez wyraźnej korzyści przy tej skali UI

TypeScript jest opcjonalny; przy dwóch–trzech osobach i małym kodzie wystarczy JSDoc z typami, który VS Code i tak sprawdza. Dane pozostają w tym samym kształcie co dziś, więc PR-y społeczności do `data.json` nadal mają sens — z zastrzeżeniem, że po wdrożeniu nocnego snapshotu źródłem prawdy jest baza, a PR do danych powinien trafiać jako zgłoszenie do operatora.

8. Koszty roczne

Ścieżka rekomendowana (PWA + Supabase Free + GitHub Pages) kosztuje 0 zł/rok, a z własną domeną ok. 60–100 zł/rok. Sklepy dokładają ok. 380 zł/rok, chyba że Okręg uzyska zwolnienie Apple dla organizacji non-profit.

Pozycja	Ścieżka 0 zł (W1)	Ze sklepami (W2)	Z gwarancjami (W2 + Supabase Pro)
Hosting strony i panelu (GitHub Pages, limit 100 GB/mies. transferu, 1 GB strony)	0	0	0
Baza danych i logowanie (Supabase)	0	0	ok. 1 140
Podkłady online (GUGiK WMTS)	0	0	0
Paczka offline na Cloudflare R2 (opcja b, do 10 GB)	0	0	0
Automatyzacje (GitHub Actions, repo publiczne)	0	0	0
Apple Developer Program	—	ok. 380 (0 przy zwolnieniu)	ok. 380 (0 przy zwolnieniu)
Google Play (jednorazowo)	—	ok. 95 raz	ok. 95 raz
Domena (subdomena <code>mapa.rzeszow.pzw.pl</code> = 0; osobna <code>.pl</code> ok. 60–100)	0–100	0–100	0–100
Razem, rok 1	0–100	ok. 475–575	ok. 1 600–1 700
Razem, kolejne lata	0–100	ok. 380–480	ok. 1 500–1 600

Kwoty w zł po kursie NBP 3,80 zł/USD (18.09.2026), zaokrąglone. Szacunek 100–150 zł/mies. z agencji spotkania można więc obniżyć do zera; jedyny wydatek, który naprawdę coś kupuje, to Apple (obecność w App Store).

9. Bezobsługowość i przekazanie PZW

„Bezobsługowość” osiąga się nie przez brak automatyzacji, lecz przez automatyzację, które w razie awarii degradują do statycznej strony, oraz przez to, że wszystkie konta należą do Okręgu, a nie do osoby.

Własność kont

Zasób	Dziś	Docelowo
Repozytorium	korntech/mapa-wod-pzw-rzeszow (konto prywatne)	Organizacja GitHub Okręgu (np. pzw-rzeszow) z 2 właścicielami: osoba z Okręgu + autor; repo przeniesione, nie skopiowane (zachowuje historię, issues, gwiazdki)
Supabase	Projekt na koncie prywatnym	Organizacja Supabase na e-mail Okręgu, autor jako członek; klucz <code>service_role</code> i hasło do bazy w sejfie haseł Okręgu
Domena	—	Subdomena <code>rzeszow.pzw.pl</code> (rekord CNAME na Pages), konfigurowana przez administratora domeny PZW
Sklepy (jeśli W2)	—	Konta na Okręg jako podmiot prawny (D-U-N-S, KRS); autor jako deweloper z uprawnieniami, nie właściciel
Kafle offline (jeśli R2)	—	Konto Cloudflare Okręgu

Automatyzacje (GitHub Actions, wszystkie w repo)

- Co noc: eksport tabel z Supabase do `data.json` i commit — backup w Git z historią zmian oraz keep-alive bazy w jednym.
- Co godzinę: healthcheck strony i bazy; przy awarii issue w repo i e-mail do wskazanego adresu Okręgu.
- Przy pushu do `main`: build Vite, testy (poprawny JSON, wszystkie punkty w granicach województwa, brak duplikatów), deploy na Pages.
- Raz w roku (przypomnienie w issue): nowy wykaz wód, przegląd zależności, odświeżenie paczki offline.

Runbook roczny dla Okręgu (bez wiedzy technicznej)

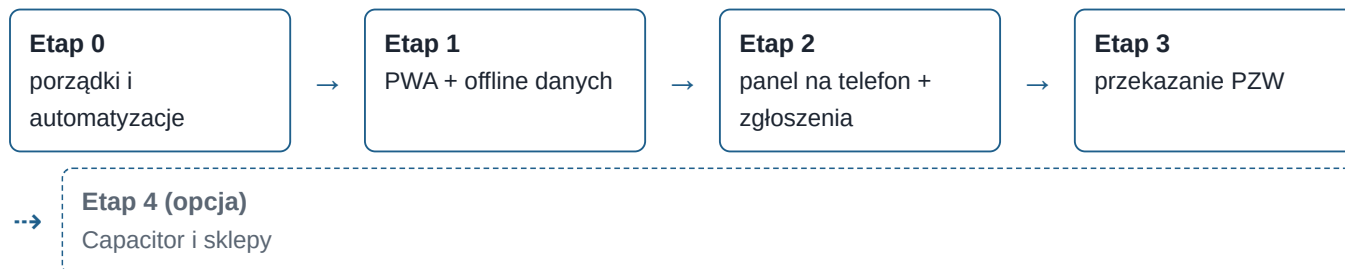
1. Styczeń: operator wprowadza zmiany z nowego wykazu w panelu (szkolenie 1 h, instrukcja z obrazkami w repo).
2. Sprawdzenie, czy przyszło przypomnienie o wygasających płatnościach (tylko domena i ewentualnie Apple).
3. Jeśli healthcheck założy issue, a nikt nie odpowie w 7 dni — mapa nadal działa ze snapshotu; naprawę może zrobić każdy z dostępem do repo.

Plan na wypadek odejścia autora

Wszystko, co potrzebne do odtworzenia systemu, jest w repo: `schema.sql`, seedy, workflowy, `README` z konfiguracją od zera. Test przekazania: druga osoba (np. z lokalnej społeczności IT lub uczelni w Rzeszowie) odtwarza środowisko z samego `README` w jeden wieczór. Dopóki ten test nie przejdzie, przekazanie nie jest zakończone.

10. Rekomendacja: PWA najpierw, gotowość na sklepy później

Budujemy jedną aplikację webową jako PWA (W1) na obecnym stosie z lekkim buildem Vite, z offline w wersji „dane zawsze, podkład na życzenie”, i utrzymujemy ją tak, by opakowanie w Capacitor (W2) było decyzją Okręgu o wydaniu ok. 380 zł/rok, a nie projektem programistycznym. React Native i Flutter odpadają: podwójny kod i krzywa nauki nie dają nic, czego potrzebuje mapa 90 obiektów.



Etap	Zakres	Kryterium gotowości
0. Porządki	Vite + moduły, biblioteki z npm, GitHub Actions: build, testy danych, deploy, nocny snapshot, healthcheck	Mapa i panel działają jak dziś; <code>data.json</code> odnawia się sam; awaria Supabase nie psuje mapy
1. PWA	Manifest, service worker, IndexedDB z danymi, ekran instalacji „dodaj do ekranu”, tryb offline (c) bez podkładu, opcjonalnie (a) „zapisz okolice”	Telefon w trybie samolotowym pokazuje łowiska, granice i zasady; Lighthouse PWA bez błędów
2. Panel mobilny	Celownik „ustaw tu”, GPS operatora, bottom sheet, większe uchwyty Geoman, ochrona przed nadpisaniem, tabela <code>zgłoszenia</code> i formularz zgłoszenia błędu dla wędkarzy	Operator poprawia lokalizację zbiornika stojąc nad nim; zgłoszenie wędkarza wysyła się po odzyskaniu sieci
3. Przekazanie	Organizacja GitHub i Supabase Okręgu, subdomena, instrukcja z obrazkami, szkolenie operatorów, test odtworzenia z README przez drugą osobę	Wszystkie konta na Okręg; druga osoba odtworzyła system; agenda spotkania odhaczona
4. Sklepy (opcja)	Capacitor, ikony i rzuty, polityka prywatności, konta deweloperskie Okręgu, wniosek o zwolnienie Apple	Aplikacja w obu sklepach; wydanie nowej wersji opisane krok po kroku w repo

Etapy 0–2 nie wymagają żadnej decyzji ani wydatku po stronie PZW i można je zacząć od razu. Godziny na każdy etap — w sekcji 12.

11. Decyzje do podjęcia

Każda pozycja ma odpowiedź domyślną — brak sprzeciwu oznacza, że idziemy według niej.

Decyzja	Kto	Propozycja domyślna
Mobile jako PWA w v1, sklepy jako opcjonalny etap 4	Kamil	Tak
Vite + moduły ES bez frameworka	Kamil	Tak; TypeScript nie, JSDoc tak
Offline: dane zawsze, podkład „zapisz okolice” z kafla GUGiK; PMTiles dopiero na życzenie użytkowników	Kamil	Tak, po sprawdzeniu regulaminu GUGiK co do buforowania
Edycja w panelu tylko online; w terenie zgłoszenia z GPS	Kamil + operatorzy	Tak
Tabela <code>zgłoszenia</code> i publiczny formularz zgłoszenia błędu (bez logowania, z limitem)	Kamil	Tak, moderacja w panelu
Organizacja GitHub i Supabase na Okręg, autor jako współwłaściciel	PZW	Do ustalenia na spotkaniu (punkt 2 agendy)
Subdomena <code>mapa.rzeszow.pzw.pl</code>	PZW	Tak, jeśli administrator domeny może dodać rekord CNAME
Czy Okręg chce obecności w App Store/Google Play i czy złoży wniosek o zwolnienie Apple	PZW	Decyzja po etapie 2, gdy PWA będzie do pokazania
Supabase Pro (ok. 1 140 zł/rok)	PZW	Nie; snapshot w Git jest backupem
Licencja danych (kod jest MIT)	PZW	Dane przestrzenne z PZGiK (BDOT10k, PRG) — bezpłatne i dostępne do ponownego wykorzystania na mocy nowelizacji Prawa geodezyjnego i kartograficznego z 2020 r.; jedyny wymóg to podanie źródła („Dane: Główny Urząd Geodezji i Kartografii, BDOT10k”). Treść merytoryczna (nazwy łowisk, powierzchnie, zasady połowu, granice obwodów) pochodzi z Wykazu wód Okręgu i jest własnością Okręgu — to Okręg decyduje o jej licencji. Brak klauzuli share-alike i zobowiązań wobec podmiotów zewnętrznych; wiążący pozostaje oficjalny wykaz
Źródła danych wyłącznie polskie (GUGiK/PZGiK)	Kamil	Tak — wykonane 20.09.2026

12. Koszt czasowy: ile godzin od Ciebie

Domknięcie projektu bez sklepów to **ok. 120–190 godzin Twojego czasu**, ze sklepami **ok. 160–250 godzin**. Praca z AI (Claude Code i podobne) skraca kodowanie mniej więcej o połowę względem ręcznego pisania, ale nie skraca spotkań, maili, czekania na decyzje Okręgu ani testów w terenie — a te stanowią jedną trzecią całości.

Założenia szacunku

- Pracujesz sam, wieczorami i w weekendy, z asystentem AI do kodu, testów, dokumentacji i przygotowania materiałów na spotkania.
- Godziny „z AI” zakładają, że AI pisze pierwszą wersję, a Ty czytasz, testujesz na telefonie i poprawiasz — to nadal Twój czas. Model nie przetestuje aplikacji nad Wisłokiem ani nie odpowie za Ciebie na maila z Okręgu.
- Widełki: dolna wartość, gdy wszystko idzie gładko i PZW odpowiada w tydzień; górna, gdy pojawiają się zmiany założeń i długie przerwy w decyzjach.
- Nie liczę czasu oczekiwania (przeglądy w sklepach, odpowiedzi z Okręgu) — tylko czas, w którym coś robisz.

Godziny według rodzaju pracy

Rodzaj pracy	Co obejmuje	Bez AI (h)	Z AI (h)	Uwagi
Etap 0 — porządki i automatyzacje	Vite, moduły, biblioteki z npm, GitHub Actions (build, testy danych, deploy, nocny snapshot, healthcheck)	30–45	12–20	AI pomaga najbardziej: workflowy i konfiguracja to szablony
Etap 1 — PWA i offline	Manifest, service worker, IndexedDB, synchronizacja przyrostowa, tryb bez podkładu, „zapisz okolicę”, testy na iPhone i Androidzie w trybie samolotowym	45–70	20–32	Najwięcej nauki (nowe API) i najwięcej testów na urządzeniach
Etap 2 — panel mobilny i zgłoszenia	Celownik, GPS operatora, bottom sheet, uchwyty Geoman, ochrona przed nadpisaniem, tabela i formularz zgłoszeń, moderacja	35–55	16–26	UX dotykowy trzeba sprawdzić ręcznie, AI nie zrobi tego za Ciebie
Etap 3 — przekazanie techniczne	Organizacje GitHub i Supabase, migracja projektu, subdomena, sejf haseł, README od zera, test odtworzenia przez drugą osobę	15–22	10–15	Głównie klikanie w panelach i czekanie na uprawnienia — AI mało pomaga
Dokumentacja dla PZW	Instrukcja operatora z obrazkami, polityka prywatności / zastrzeżenie, notatka o utrzymaniu, materiały na spotkania	15–22	6–10	AI pisze, Ty robisz zrzuty ekranu i sprawdzasz
Spotkania z Okręgiem	Spotkanie wg agendy (2–3 h) + 2–4 kolejne (pokaz PWA, szkolenie operatorów 1 h, przekazanie), każde z przygotowaniem i dojazdem	14–24	12–20	AI skraca tylko przygotowanie
Maile i koordynacja	Ustalenia z Okręgiem, administratorem domeny, operatorami; ok. 30–45 min tygodniowo przez 5–8 miesięcy	12–24	10–20	Zależy od tempa odpowiedzi po stronie PZW
Zmiany założeń architektonicznych	Po pierwszym spotkaniu i po pokazie PWA: typowo 2–3 korekty (np. inne uprawnienia operatorów, inny zakres offline, inna domena)	15–30	8–16	Wliczone przerabianie już napisanego kodu
Testy w terenie i z operatorami	2–3 wyjścia nad wodę z telefonem, sesja z operatorem na jego telefonie, poprawki po testach	10–16	10–16	Nieskracalne
Wsparcie po starcie (pierwsze 3 miesiące)	Błędy zgłaszane przez wędkarzy, pytania operatorów, drobne poprawki danych	12–24	8–16	Potem powinno spaść do kilku godzin rocznie
Razem bez sklepów		203–332	112–191	zaokrąglając: 120–190 h

Etap 4 — sklepy (opcja)	Capacitor, nauka Xcode/Android Studio, ikony i zrzuty, konta Okręgu (D-U-N-S, wniosek o zwolnienie Apple), pierwsza publikacja, poprawki po odrzuceniu, instrukcja wydawania	45–80	30–50	Plus czekanie: przeglądy 1–7 dni, weryfikacja konta organizacji 1–3 tygodnie
Zmiany założeń wynikające ze sklepów	Polityka prywatności, wymagania Apple co do logowania i lokalizacji, dodatkowe ustalenia z Okręgiem jako właścicielem kont	8–14	6–10	
Razem ze sklepami		256–426	148–251	zaokrąglając: 160–250 h

Ile to w kalendarzu

Tempo	Bez sklepów	Ze sklepami	Uwagi
5 h/tydzień (jeden wieczór + kawałek weekendu)	6–9 miesięcy	8–12 miesięcy	Realistyczne przy pracy zawodowej; przerwy na decyzje PZW mieszczą się „za darmo”
10 h/tydzień	3–5 miesięcy	4–6 miesięcy	Wąskim gardłem staje się Okręg, nie Ty
Etapy 0–2 same (bez PZW)	50–80 h $\approx$ 5–8 tygodni przy 10 h/tydz.	—	Można zacząć jutro; do pokazania na pierwszym spotkaniu

Co najbardziej rozciąga czas — i jak temu zapobiec

- **Decyzje Okręgu bez terminu.** Każdy punkt agendy dostaje proponowaną odpowiedź domyślną i datę, po której wchodzi w życie (sekcja 11). To zamienia „czekamy” na „idziemy, chyba że ktoś się sprzeciwi”.
- **Zmiany założeń po napisaniu kodu.** Pokaż działającą PWA (koniec etapu 1) zanim zaczniesz panel mobilny; zmiany są wtedy tańsze o połowę.
- **Sklepy.** Prawie jedna trzecia dodatkowego czasu i jedyny element z powtarzalnym obowiązkiem rocznym. Dlatego są etapem 4, a nie warunkiem startu.
- **Testy na iOS.** Safari zachowuje się inaczej niż Chrome; zaplanuj testy na prawdziwym iPhone od etapu 1, nie na końcu.

Przejście na źródła BDOT10k dokłada do etapu 0 ok. 8–14 godzin: ekstraktor danych (pobranie paczek per powiat, wycięcie warstw rzek, kanałów i poligonów wód), podmiana geometrii w bazie oraz weryfikacja, czy nowe kontury i powierzchnie zgadzają się z wykazem. To jednorazowy koszt, który zdejmuje z projektu ryzyko licencyjne i zamyka temat na stałe — zamiast wracać do niego przy każdym pytaniu o pochodzenie danych. Prace te zostały wykonane 20.09.2026, więc ten czas jest już za nami i nie obciąża pozostałego harmonogramu.

13. Źródła

- Apple Developer Program, opłata roczna: developer.apple.com/support/compare-memberships; zwolnienie z opłaty dla organizacji non-profit: developer.apple.com/support/membership-fee-waiver
- Google Play, opłata rejestracyjna: support.google.com; konto organizacji i reguła 12 testerów: iconikai.com
- Supabase, plany Free i Pro: supabase.com/pricing
- GitHub Pages, limity: docs.github.com
- Cloudflare R2, cennik: developers.cloudflare.com/r2/pricing
- Expo EAS, cennik: expo.dev/pricing
- Capacitor, wymagania środowiska: capacitorjs.com
- Usługi pobierania WFS Geoportalu: geoportal.gov.pl/en/services/download-service-wfs
- BDOT10k, Baza Danych Obiektów Topograficznych GUGiK: gugik.gov.pl/pzgik/zamow-dane/baza-danych-obiektow-topograficznych-bdot-10k
- UUG — Uniwersalna Usługa Geokodowania GUGiK: services.gugik.gov.pl/uug
- Polityka korzystania z kafli OpenStreetMap (zakaz pobierania na zapas — uzasadnienie usunięcia fallbacku): operations.osmfoundation.org/policies/tiles
- WebKit, zwolnienie PWA z 7-dniowego kasowania danych: webkit.org/tracking-prevention; limity pamięci: webkit.org/blog/14403
- Apple wycofuje blokadę PWA w UE (iOS 17.4): techcrunch.com
- Kurs USD/PLN: NBP, tabela A z 18.09.2026, 3,7998 zł
- Stan projektu: README, specyfikacje w `docs/superpowers/specs/` i `agenda-spotkania-pzw.md` w repozytorium